

Does explicit screen structure substitute for scale?

OCR-augmented small vision–language models on GUI grounding

Alienor Sabourdin

AIDAMS RS Project · June–August 2026

Abstract

Vision–Language Models (VLMs) are models that can understand both text and images and generate a textual output. Such tools are keys for building accessibility tools or interactive agents. Today, the strongest VLMs present highly capabilities, but they are very large, costly and rarely open-source, while the smaller VLMs, which can run locally, have usually poor performances on complicated tasks. So users or companies wanting to run a small VLM could benefit from strategies to better their performances. VLMs can run on all types of images, real pictures or digital images. The case of screenshots is particularly interesting. Indeed, screenshots are “digital images”: contrarily to a photograph taken in real life, they are deterministically rendered and dominated by text, so an important part of their content is explicit and cheap to extract. Thus, we ask whether supplying that structure to a small model as an inductive bias can increase its performances and possibly compensate for scale. We evaluate Qwen2.5-VL at 3B and 7B parameters on the ScreenSpot grounding benchmark under three input conditions: the plain screenshot, the screenshot plus the full OCR (text-and-bbox list), and the screenshot plus only the OCR lines matching the instruction. The rest of the pipeline stays identical across all six scenarios. We measure click accuracy by element type (text or icon), alongside latency and a filter diagnostic. Contrary to our initial expectation that structure would better the small model’s performance, the full OCR list collapsed the 3B’s accuracy from 0.829 to 0.296: the model drifts toward wrong elements even when the correct target is present in the extracted text. The damage is largest on text targets, inverting the usual text-easier-than-icons ordering. Filtering the structure to the instruction recovers some of the loss (to 0.675), allowing us to localize the harm to distraction by volume. However, providing the OCR never improved the 3B model above its plain baseline. Explicit structure as extracted is thus not a free substitute for scale but an extra input a model must be large enough to use. Nonetheless, other ways of providing structure are left unexplored, and feeding only relevant structure still appears as a promising direction, especially since the 7B model’s textual click accuracy improved, although insignificantly, from 0.909 to 0.955.

1 Introduction

Screen understanding and parsing is used in a growing number of tools such as robotic process automation or accessibility software. Today’s main way to treat such images is to feed them to a general vision–language model, trained on many different images, including screenshots but also real pictures. Such models have billions of parameters, and therefore are expensive to use, take a lot of memory and are most of the time only accessible through companies’ cloud services. Their local use is therefore difficult, often impossible and pretty slow. There exist smaller VLMs that run locally, but they perform substantially worse, especially on images containing a lot of information such as screenshots.

However, screenshots are very different from natural photographs. They are images of pixels produced by a specific, deterministic rendering process and can be decomposed into different regions, each serving a specific purpose. They are dominated by text or specific icons, and cannot contain visual artefacts or be blurry. These properties are visible, and can sometimes be cheap to recover with dedicated tools, whereas a generic VLM must learn them implicitly from data, using many of its parameters. But what if that information were supplied directly in the prompt instead of learned in the weights? This motivated this study’s question:

Can an explicit structural signal (OCR-extracted text and positions) let a small VLM match a larger model of the same

family on GUI grounding?

We test this through a straightforward pipeline, comparing two models of different size from the same family (Qwen2.5-VL 3B and 7B), so as not to confound scale with genuine architectural differences, although the latter would be interesting for future work. We test these models under three different scenarios: the plain screenshot, the screenshot with all its extracted text, and the screenshot with only the relevant elements of its extracted text. We provide through this work: (i) a reproducible pipeline to test these models, (ii) an evaluation of the models with and without OCR add-ons, (iii) a discussion of these results and of trajectories for future work.

2 Related work

GUI grounding benchmarks. ScreenSpot (introduced with SeeClick) [1] is a grounding benchmark. It evaluates the ability of a model to locate a target element in an image, given a natural-language instruction. It comes with a dataset spanning mobile, desktop, and web interfaces, with targets labelled as either text or icons. Prior work reports that general VLMs underperform GUI-specialised ones on this task, and follow-up benchmarks such as ScreenSpot-Pro [2] show accuracy collapsing on high-resolution UIs with tiny targets.

Small vision–language models. Qwen2.5-VL [3] processes images at dynamic resolution and is trained to emit grounding

coordinates. It is available at several sizes (3B, 7B, 72B). Newer families (e.g. Qwen3-VL) extend the small-model range further and could be the object of future testing.

Structural augmentation. Explicit structure has repeatedly been shown to aid screen reasoning. Screen parsers such as OmniParser [4] convert a screenshot into a list of labelled, localised elements for agents, and Set-of-Mark prompting [5] shows that overlaying region marks improves large-VLM grounding. Our study differs from these because it uses structure not to build an agent or answer a specific task, but as a way to improve general screenshot understanding.

3 Method

3.1 Task and metric

We evaluate our models on ScreenSpot. The metric is the standard ScreenSpot accuracy: a prediction is a single click point, correct if and only if it falls inside the ground-truth bounding box. We report overall accuracy and accuracy split by element type (text/icon). We also measure mean inference latency and, for the structural conditions, mean OCR time.

3.2 Models

We use Qwen2.5-VL-3B and Qwen2.5-VL-7B, both 4-bit, running locally on Apple Silicon via `mlx-vlm` [6]. Because the two share architecture and training recipe, differences between them only result from scale. We set the models’ temperature to 0 for determinism.

3.3 Structural signal and the three conditions

We compare three input conditions (plain, structure and filtered), holding everything else strictly identical. In **plain**, the model sees only the screenshot. In **structure**, we run RapidOCR [7] (a self-contained ONNX OCR engine, chosen for performance and reproducibility) on the image the model sees and inject the *full* list of detected text strings with pixel boxes, rendered in reading order as "text" [x1,y1,x2,y2] per line. In **filtered**, we inject only the OCR lines whose text overlaps the instruction: each line is scored by shared tokens with the instruction (plus a substring bonus), and we keep the top few (≤ 5); if nothing matches, the block is empty. The filtered condition tests our diagnosis that the small model is harmed by the *volume* of unfiltered structure rather than by structure as such. All three conditions share the same prompt template and output format, so any difference is attributable to the injected block alone.

3.4 Coordinate handling

The main technical pitfall in grounding is the coordinate convention. Qwen2.5-VL emits pixel coordinates in the space of

its internally *smart-resized* image [8]. We therefore pre-resize every screenshot with Qwen’s own `smart_resize` to a known resolution, feed that image to the model, read predicted pixels in that space, and rescale linearly back to the original image before scoring. Crucially, this convention is **identical across all six cells**, so even an error in the absolute mapping cannot bias the comparisons between conditions or sizes. We validate the mapping before the full run with a smoke test that overlays predicted points and ground-truth boxes on sample images.

3.5 Protocol

We evaluate a fixed, stratified subset of **240** examples (seeded for reproducibility), preserving the benchmark’s text/icon and platform balance, and use the *same* subset for every cell (2 sizes $\times$ 3 conditions = 6 evaluations). Each image is resized once and OCR’d once, with the OCR result cached and shared by the structure and filtered conditions and across both models. All code and the exact subset construction are available in the repository.

4 Results

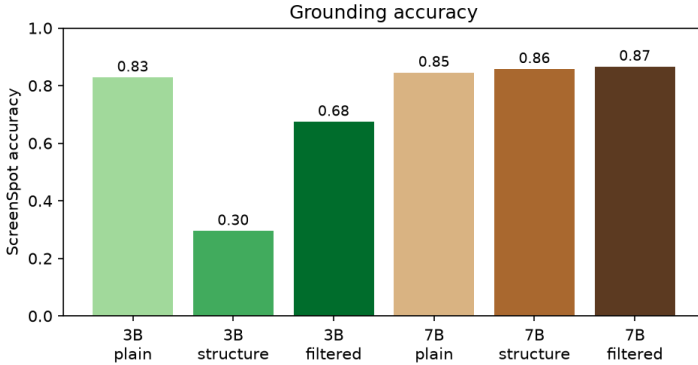
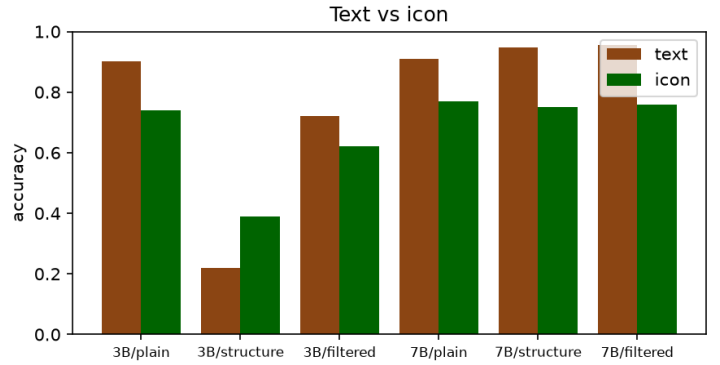
Numbers are produced by `src/analyze.py` from a single seeded run of 240 examples per cell; parse-failure rate is 0.00 in every cell, so all differences reflect grounding, not output-format artefacts.

Full structure breaks the small model, not the large one (Figure 1). Injecting the full OCR list moves 7B accuracy by only +0.012 (0.846 to 0.858) but moves 3B accuracy by -0.533 (0.829 to 0.296) (Table 1). Both models receive a byte-identical signal on the same images, so the divergence is attributable to capacity, not to the signal’s quality. On the 3B the damage is on text targets, where the OCR block is more important: -0.682 on text (0.902 to 0.220) versus -0.352 on icons. This inverts the natural ordering: in every plain cell text is easier than icons, but under full structure the 3B’s text accuracy (0.220) falls below its icon accuracy (0.389). Case-level inspection (`diagnose_structure.py`) shows why: with 15 to 45 OCR lines present, the 3B is distracted from the instructed target, even when its exact string is in the list, and selects a generically salient element instead (a primary call-to-action button, or a hallucinated icon).

Filtering makes it slightly better, but stays below the baseline. Restricting the block to the OCR lines matching the instruction lifts the 3B from 0.296 back to 0.675 (+0.379 over full structure) and restores the natural ordering (text 0.720 > icon 0.620). This confirms that the volume of unfiltered structure drove most of the accuracy drop. However, filtered accuracy (0.675) is still lower than plain (0.829) on every breakdown: even a short, on-topic block of positional text does not lift the 3B model above what it already achieves from pixels alone. Figure 2 isolates this by element type: under *structure* the 3B’s text bar drops below

Table 1: Grounding accuracy and cost (2 sizes $\times$ 3 conditions, $n = 240$ per cell).

Model	Condition	Accuracy	Text	Icon	Latency (s)	OCR (s)
3B	plain	0.829	0.902	0.741	8.5	—
3B	structure	0.296	0.220	0.389	11.7	0.76
3B	filtered	0.675	0.720	0.620	9.6	0.76
7B	plain	0.846	0.909	0.769	15.5	—
7B	structure	0.858	0.947	0.750	20.3	0.76
7B	filtered	0.867	0.955	0.759	15.9	0.76

**Figure 1:** ScreenSpot click accuracy for each model under the three conditions. The same OCR signal collapses the 3B under *structure* (-0.533) yet leaves the 7B essentially flat; *filtered* recovers most of the 3B loss without reaching plain.**Figure 2:** 3B and 7B accuracy by element type across the three conditions. Full *structure* inverts the usual text-over-icon ordering; *filtered* restores it but not the plain level.

its icon bar (the inversion), and *filtered* pulls both back up and restores text above icon, though neither returns to the plain level.

Filtered structure results in a slight improvement in the 7B’s accuracy. For the 7B the three conditions are slightly monotonically positive (plain 0.846, structure 0.858, filtered 0.867). It is particularly interesting that the filtered structure gains $+0.046$ over plain on text. This could indicate that for a mid-sized model (7B is runnable locally on most computers) – a model with the capacity to use structure – it can gain a little from it, especially when it is exactly relevant to the instruction.

Why filtering partly recovers. Filtering has two effects on the provided structure. First, filter recall: the instruction-matching verification kept the target element’s own OCR line in only 53% of examples (126/240). Second, residual cost: even when the target line was kept, 3B/filtered reached just 0.730, still below the 0.829 plain baseline, whereas when it was dropped accuracy fell to 0.614 (Figure 3). So better filtering would help but it cannot fully close the gap to plain: injected text carries an important cost for the small model. It is also worth noting that the filter recall mostly kept the extracted text of textual targets, as icons have fewer matching tokens, so the two subsets are not evenly measured across target types.

Time cost. OCR adds around 0.76 s per image. Full structure also lengthens the prompt, raising 7B latency from 15.5 to 20.3 s. Filtering keeps the block short, so 3B/filtered (9.6 s) is barely above plain (8.5 s). For the small model the full structure

condition is thus both slower and less accurate.

5 Discussion and limitations

Our starting hypothesis — that explicit structure could substitute for scale — is decisively not supported, and the three-condition design turns the negative into a mechanism. The same OCR signal that the 7B absorbs ($+0.012$) collapses the 3B (-0.533), so structure is not a free context window but an *additional input the model must have the capacity to integrate*. Filtering the signal to the instruction recovers most of the small-model loss ($0.296 \rightarrow 0.675$), which localises the bulk of the harm to *distraction by volume* rather than to structure as such: a model given a short, on-topic list is far less derailed than one given the whole screen. Yet filtered structure never lifts the 3B above its own plain baseline ($0.675 < 0.829$), and even when the correct line is kept it reaches only 0.730: for a small model, position-annotated text is at best a liability to be minimised, not a source of gain. Structure therefore *requires* scale before it pays off. This reframes the on-device question: the lever for small models is not to add structure but to add as little as possible and target it precisely — and even then to expect neutral-to-negative returns, with the real headroom in better instruction-to-element retrieval (our filter kept the right line only 53% of the time).

The structural signal is cheap, model-agnostic, interpretable, and needs no training; on the larger model it is neutral-to-slightly-helpful ($0.846 \rightarrow 0.867$ when filtered) at a fixed ~ 0.76 s of

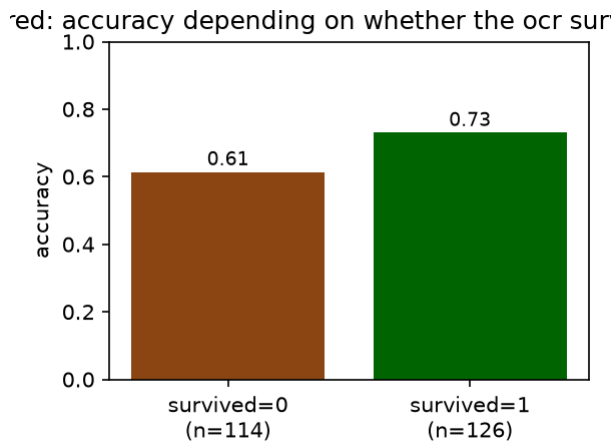


Figure 3: 3B/filtered accuracy split by whether the target’s own OCR line survived the filter. Both bars sit below the plain baseline, so recall explains part, but not all, of the residual gap.

OCR. Filtering is a simple, effective mitigation of the small-model failure and keeps the prompt (and its latency) short.

These results also indicate several problems with adding structure as OCR-extracted text to VLM prompts:

- Unfiltered structure *distracts* a very small model catastrophically.
- Even filtered, structure does not help the small model beyond its plain baseline.

There are also limitations to our approach:

- Our filter is a simple keyword heuristic with 53% target recall; a better retriever would probably change the filtered numbers.
- The extraction step depends on OCR quality, which degrades on dense toolbars and stylised text.
- We recover text positions but not higher-order structure (layout grouping, z-order/occlusion).
- We evaluate one model family at 4-bit on a stratified 240-example subset, not the full benchmark.
- We do not compare against a frontier cloud VLM, so we speak to the scale-vs-structure interaction *within* a family.
- We test one delivery format (a text list in the prompt); marks drawn on the image might behave differently.

This study and its limitations open several trajectories for future work. Because harm scales with the amount of injected structure, a stronger instruction-to-element *retriever* (for instance semantic rather than keyword) should raise the filtered numbers towards the plain baseline. It would be interesting to replicate the experiment on a newer family (Qwen3-VL) to test whether larger small-models absorb structure better, as slightly suggested

by the 7B improvement (though not statistically relevant here). Another test could be marks-on-image instead of a text list, to separate hints in the context from actual image structure. It would also be interesting to extract other elements, such as UI-specific structure, to help with icon understanding. Finally, evaluating these augmentations on ScreenSpot-Pro — a dense, high-resolution regime where the size gap is largest — would be valuable.

6 Conclusion

We tested whether explicit, OCR-derived structure lets a small VLM compete with a larger one on GUI grounding, comparing two models from the same family at different sizes under three input conditions on a public grounding benchmark. The result is a clean negative with a mechanism: the identical OCR signal left the 7B model unchanged (+0.012) but broke the 3B model (−0.533), which drifted from correct-but-listed targets toward wrong elements, most severely on text-dense screens. Filtering the structure to the instruction recovered some of the small-model loss (to 0.675), localising the harm to distraction by volume. The main finding is that explicit structure is not a free substitute for scale: it is an extra input a model must be large enough to use, and for small models the productive question is not *whether* to add structure but *how much* and *how precisely targeted*. A negative result against our initial hypothesis, but a clear, mechanistic, and useful one.

Use of LLM assistance. Portions of the code, and the structure and review of this paper, were produced with LLM assistance. The process is documented and was reviewed line by line. The pipeline design, choice of tools and metrics were my own.

References

- [1] K. Cheng, Q. Sun, Y. Chu, F. Xu, Y. Li, J. Zhang, and Z. Wu. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. *arXiv:2401.10935*, 2024.
- [2] K. Li et al. ScreenSpot-Pro: GUI Grounding for Professional High-Resolution Computer Use. *arXiv:2504.07981*, 2025.
- [3] S. Bai et al. Qwen2.5-VL Technical Report. *arXiv:2502.13923*, 2025.
- [4] Y. Lu, J. Yang, Y. Shen, and A. Awadallah. OmniParser for Pure Vision Based GUI Agent. *arXiv:2408.00203*, 2024.
- [5] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao. Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. *arXiv:2310.11441*, 2023.
- [6] P. Canuma. mlx-vlm: Vision Language Models on Apple Silicon with MLX. <https://github.com/Blaizzy/mlx-vlm>, 2024.
- [7] RapidAI. RapidOCR. <https://github.com/RapidAI/RapidOCR>, 2023.
- [8] P. Wang, S. Bai, S. Tan, et al. Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution. *arXiv:2409.12191*, 2024.